Expert Systems With Applications 234 (2023) 121053

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Geometric SMOTE for imbalanced datasets with nominal and continuous features

Joao Fonseca *, Fernando Bacao

NOVA Information Management School, Universidade Nova de Lisboa, Campus de Campolide, Lisboa, 1070–312, Portugal

ARTICLE INFO

Keywords:
Imbalanced learning
Oversampling
SMOTE
Data generation
Nominal data

ABSTRACT

Imbalanced learning can be addressed in 3 different ways: Resampling, algorithmic modifications and cost-sensitive solutions. Resampling, and specifically oversampling, are more general approaches when opposed to algorithmic and cost-sensitive methods. Since the proposal of the Synthetic Minority Oversampling TEchnique (SMOTE), various SMOTE variants and neural network-based oversampling methods have been developed. However, the options to oversample datasets with nominal and continuous features are limited. We propose Geometric SMOTE for Nominal and Continuous features (G-SMOTENC), based on a combination of G-SMOTE and SMOTENC. Our method modifies SMOTENC's encoding and generation mechanism for nominal features while using G-SMOTE's data selection mechanism to determine the center observation and k-nearest neighbors and generation mechanism for continuous features. G-SMOTENC's performance is compared against SMOTENC's along with two other baseline methods, a State-of-the-art oversampling method and no oversampling. The experiment was performed over 20 datasets with varying imbalance ratios, number of metric and non-metric features and target classes. We found a significant improvement in classification performance when using G-SMOTENC as the oversampling method. An open-source implementation of G-SMOTENC is made available in the Python programming language.

1. Introduction

Various Machine Learning (ML) tasks deal with highly imbalanced datasets, such as fraud transactions detection, fault detection and medical diagnosis (Tyagi & Mittal, 2020). In these situations, predicting false positives is often a more acceptable error, since the class of interest is usually the minority class (Vuttipittayamongkol, Elyan, & Petrovski, 2021). However, using standard ML classifiers on imbalanced datasets induces a bias in favor of the classes with the highest frequency, while limiting the predictive power on lower frequency classes (Das, Datta, & Chaudhuri, 2018; López, Fernández, García, Palade, & Herrera, 2013). This effect is known in the ML community as the Imbalanced Learning problem.

Imbalanced learning involves a dataset with two or more target classes with varying class frequencies. The minority class is defined as the class with the least amount of observations and the majority class is the one with the highest amount of observations (Kaur, Pannu, & Malhi, 2019). There are three main approaches to address imbalanced learning (Fernández, LóPez, Galar, Del Jesus, & Herrera, 2013):

1. Cost-sensitive solutions attribute a higher misclassification cost to the minority class observations to minimize higher cost errors;
2. Algorithmic level solutions modify ML classifiers to improve the learning of the minority class;
3. Resampling solutions generate synthetic minority class observations and/or remove majority class observations to balance the training dataset;

Since it is an external approach to imbalanced learning, the latter method becomes particularly useful. It dismisses the required domain knowledge to build a cost matrix and the technical complexity or knowledge to apply an imbalanced learning-specific classifier. Resampling can be done via undersampling, oversampling, or hybrid approaches (Tarekegn, Giacobini, & Michalak, 2021). In this paper, we will focus on oversampling approaches.

The presence of nominal features in imbalanced learning tasks limits the options available to deal with class imbalance. Even though it is possible to use encoding methods such as one-hot or ordinal encoding to convert nominal features into numerical, applying a distance metric on mixed-type datasets is questionable since the nominal feature values are unordered (Lumijärvi, Laurikkala, & Juhola, 2004). In this case, one possible approach is to use models that can handle different scales (*e.g.*, Decision Tree). However, this assumption may be limiting

* Corresponding author.
E-mail addresses: jpfonseca@novaims.unl.pt (J. Fonseca), bacao@novaims.unl.pt (F. Bacao).

https://doi.org/10.1016/j.eswa.2023.121053
Received 12 September 2022; Received in revised form 25 July 2023; Accepted 25 July 2023
Available online 28 July 2023

since there are few ML algorithms where this condition is verified. Another possible approach is transforming the variables to meet scale assumptions (Lumijärvi et al., 2004). This method was explored in the algorithm Synthetic Minority Oversampling Technique for Nominal and Continuous features (SMOTENC) (Chawla, Bowyer, Hall, & Kegelmeyer, 2002) (explained in Section 2).

In the presence of datasets with mixed data types, using most of the well-known resampling algorithms becomes unfeasible. This happens because these methods consider exclusively continuous data; they were not adapted to also use nominal features. Specifically, since the proposal of SMOTE, various other SMOTE-variants have been developed to address some of its limitations. Although, there was not a significant development in research to oversample datasets with both nominal and continuous features.

In this paper, we propose Geometric SMOTE for Nominal and Continuous features (G-SMOTENC). It generates the continuous feature values of a synthetic observation within a truncated hyper-spheroid with its nominal feature values using the most common value of its nearest neighbors. In addition, G-SMOTENC uses G-SMOTE's data selection strategy and SMOTENC's approach to find the center observation's nearest neighbors. G-SMOTENC is a generalization of both SMOTENC and G-SMOTE (Douzas & Bacao, 2019). With the correct hyperparameters, our G-SMOTENC implementation can mimic the behavior of SMOTE, SMOTENC, or G-SMOTE. It is available in the https://github.com/joaopfonseca/ml-researchopen-source Python library "ML-Research" and is fully compatible with the Scikit-Learn ecosystem. These contributions can be summarized as follows:

1. We propose G-SMOTENC, an oversampling algorithm for datasets with nominal and continuous features;
2. We test the proposed oversampler using 20 datasets and compare its performance to SMOTENC, Random Oversampling, Random Undersampling and a State-of-the-art oversampler;
3. We provide an implementation of G-SMOTENC in the Python programming language;

The rest of this paper is structured as follows: Section 2 describes the related work and its limitations, Section 3 describes the proposed method (G-SMOTENC), Section 4 lays out the methodology used to test G-SMOTENC, Section 5 shows and discusses the results obtained in the experiment and Section 6 presents the conclusions drawn from this study.

2. Related work

A classification problem contains n classes, having C_{maj} as the set of majority class observations (*i.e.*, observations belonging to the most common target class) and C_{min} as the set of minority class observations (*i.e.*, observations belonging to the least common target class). Typically, an oversampling algorithm will generate synthetic data in order to ensure $|C'_{min}| = |C_{maj}| = |C_i|, i \in \{1, \dots, n\}$.

Since the proposal of SMOTE, other methods modified or extended SMOTE to improve the quality of the data generated. The process of generating synthetic data using SMOTE-based algorithms can be divided into two distinct phases (Fernández, Garcia, Herrera, & Chawla, 2018):

1. Data selection. A synthetic observation, x^{gen}, is generated based on two existing observations. A SMOTE-based algorithm employs a given heuristic to select a non-majority class observation as the center observation, x^c, and one of its nearest neighbors, x^{nn}, selected randomly. For the case of SMOTE, x^c is randomly selected from each non-majority class.
2. Data generation. Once x^c and x^{nn} have been selected, x^{gen} is generated based on a transformation between the two selected observations. In the case of SMOTE, this transformation is a linear interpolation between the two observations: $x^{gen} = \alpha x^c + (1-\alpha)x^{nn}, \alpha \sim \mathcal{U}(0,1)$.

Modifications to the SMOTE algorithm can be distinguished according to the phase where they were applied. This distinction is especially relevant for the case of oversampling on datasets with mixed data types since it raises the challenge of calculating meaningful distances and k-nearest neighbors among observations. For example, State-of-the-art oversampling methods, such as Borderline-SMOTE (Han, Wang, & Mao, 2005), ADASYN (He, Bai, Garcia, & Li, 2008), K-means SMOTE (Douzas, Bacao, & Last, 2018) and LR-SMOTE (Liang, Jiang, Li, Xue, & Wang, 2020) modify the data selection mechanism and show promising results in imbalanced learning (Fonseca, Douzas, & Bacao, 2021a). However, these algorithms select x^c using procedures that include calculating each observation's k-nearest neighbors or clustering methods, which are not prepared to handle nominal data.

Modifications to SMOTE's generation mechanism are uncommon. A few oversampling methods, such as Safe-level SMOTE (Bunkhumpornpat, Sinapiromsaran, & Lursinsap, 2009) and Geometric-SMOTE (Douzas & Bacao, 2019) proposed this type of modification and have shown promising results (Douzas, Bacao, Fonseca, & Khudinyan, 2019). However, these methods are also unable to handle datasets with nominal data. Other methods attempt to replace the SMOTE data generation mechanism altogether using different Generative Adversarial Networks (GAN) architectures (Jo & Kim, 2022; Koivu, Sairanen, Airola, & Pahikkala, 2020; Salazar, Vergara, & Safont, 2021). Network-based architectures, however, are computationally expensive to train and sensitive to the training initialization. It is also difficult to ensure a balanced training of the two networks involved and tuning their hyperparameters is often challenging or unfeasible (Gonog & Zhou, 2019).

As discussed in Section 1, research on resampling methods with mixed data types is scarce. The original paper proposing SMOTE also proposed SMOTE for Nominal and Continuous (SMOTENC), an adaptation of SMOTE to handle datasets with nominal and continuous features (Chawla et al., 2002). To determine the k-nearest neighbors of x^c, the Euclidean distance is modified to include the median of the standard deviations of the continuous features for every nominal feature with different values. Once x^c and x^{nn} are defined, the continuous feature values in x^{gen} are generated using the SMOTE generation mechanism. The nominal features are given the most common values occurring in the k-nearest neighbors.

Recently, a new SMOTE-based oversampling method for datasets with mixed data types, SMOTE-ENC (Mukherjee & Khushi, 2021), was proposed. This method modifies the encoding mechanism for nominal features used in the SMOTENC algorithm to account for nominal features' change of association with minority classes. The Multivariate Normal Distribution-based Oversampling for Numerical and Categorical features (MNDO-NC) (Ambai & Fujita, 2019) uses the original MNDO method (Ambai & Fujita, 2018) along with the SMOTENC encoding mechanism to find the values of the categorical features for the synthetic observation. However, the results reported in the paper showed that MNDO-NC was consistently outperformed by SMOTENC, which led us to discard this approach from further consideration.

Alternatively to SMOTE-based methods, it is possible to use non-informed over and undersampling methods for datasets with nominal and continuous features, specifically Random Oversampling (ROS) and Random Undersampling (RUS). These methods consist of randomly duplicating minority class observations (in the case of ROS), which can lead to overfitting (Batista, Prati, & Monard, 2004; Park & Park, 2021), or randomly removing majority class observations (in the case of RUS), which may lead to underfitting (Bansal & Jain, 2021).

3. Proposed method

We propose G-SMOTENC to oversample imbalanced datasets with both nominal and continuous features. Our method builds on top of G-SMOTE's selection and generation mechanisms coupled with a modified version of SMOTENC. It attributes less importance to the nominal

features (relative to the continuous features) when computing distances among observations compared to SMOTENC. However, this method can be extended with further modifications to the nominal data encoding and selection mechanisms in future work.

Similar to G-SMOTE being an extension of SMOTE, G-SMOTENC is also an extension of SMOTENC since any method or ML pipeline using the SMOTENC generation mechanism can replace it with G-SMOTENC without any further modifications. The proposed method is described in pseudo-code in Algorithm 1. The functions $SelectionMechanism$ and $GenerationMechanism$ are described in Algorithms 2 and 3, respectively.

Algorithm 1: G-SMOTENC.

Given: Dataset with binary target classes C_{min} and C_{maj}
Input: $C_{maj}, C_{min}, \alpha_{sel}, \alpha_{trunc}, \alpha_{def}$
Output: C^{gen}
begin
  $N \leftarrow |C_{maj}| - |C_{min}|$
  $C^{gen} \leftarrow \emptyset$
  while $|C^{gen}| < N$ **do**
    $x^c, x^{nn}, X^{nn} \leftarrow SelectionMechanism(C_{maj}, C_{min}, \alpha_{sel})$
    $x^{gen} \leftarrow GenerationMechanism(x^c, x^{nn}, X^{nn}, \alpha_{trunc}, \alpha_{def})$
    $C^{gen} \leftarrow C^{gen} \cup \{x^{gen}\}$

G-SMOTENC's implementation involves additional considerations regarding the management of the nominal features. During the selection mechanism (identified as the function $SelectionMechanism$), the nominal features are encoded using the one-hot encoding technique, while the non-zero constant assumes the value of the median of the standard deviations of the continuous features in C_{min}, divided by two. This encoding mechanism varies from the one in SMOTENC in order to attribute less weight to the nominal features relative to the continuous features.

The selection strategy, α_{sel}, as well as C_{min} and C_{maj} are used to determine a central observation, x^c, its nearest neighbors, X^{nn}, and one of its nearest neighbors, $x^{nn} \in X^{nn}$. X^{nn} is calculated using the euclidean distance and both the continuous and encoded nominal features. The outcome of this step is dependent on the choice of α_{sel}:

1. If $\alpha_{sel} = minority$, X^{nn} will consist of x^c's k-nearest neighbors within C_{min};
2. If $\alpha_{sel} = majority$, X^{nn} will consist of x^c's nearest neighbor within C_{maj};
3. If $\alpha_{sel} = combined$, X^{nn} will consist of the union between x^c's k-nearest neighbors within C_{min} and x^c's nearest neighbor within C_{maj}, *i.e.*, $C_{min,k} \cup C_{maj,1}$. In this case, x^{nn} is selected using the majority class observation within X^{nn} as well as another randomly selected nearest neighbor, such that $x^{nn} = argmin(\|x^{nn}_{min} - x^c\|, \|x^{nn}_{maj} - x^c\|)$;

Unlike in the original G-SMOTE generation mechanism, X^{nn} is used in the to determine the nominal feature values of x^{gen} based on the mode of these features within X^{nn}. G-SMOTENC's generation mechanism (identified as $GenerationMechanism$) uses two hyperparameters to generate the continuous features in x^{gen}: the truncation factor, α_{trunc}, and the deformation factor, α_{def}. They are generated by forming a hyper-sphere with center x^c and is modified according to the parameters:

1. α_{trunc} truncates the hyper-sphere to induce the generation of the artificial instance within a subset of the hypersphere. It varies between 1 and -1, where 1 would split the generation area in half and use the area between x^c and x^{nn}, -1 achieves the same effect and uses the other semi-hyper-sphere, and 0 applies no truncation.
2. α_{def} deforms the hyper-sphere as shown in Fig. 1. It varies between 0 and 1, where 0 applies no deformation and 1 fully deforms the hyper-sphere into a line segment, corresponding to $e^{//}$.

Fig. 1 depicts the effect of those hyperparameters in the data selection and generation phases. For an in-depth explanation of these hyperparameters, the reader is referred to Douzas and Bacao (2019).

3.1. Selection mechanism

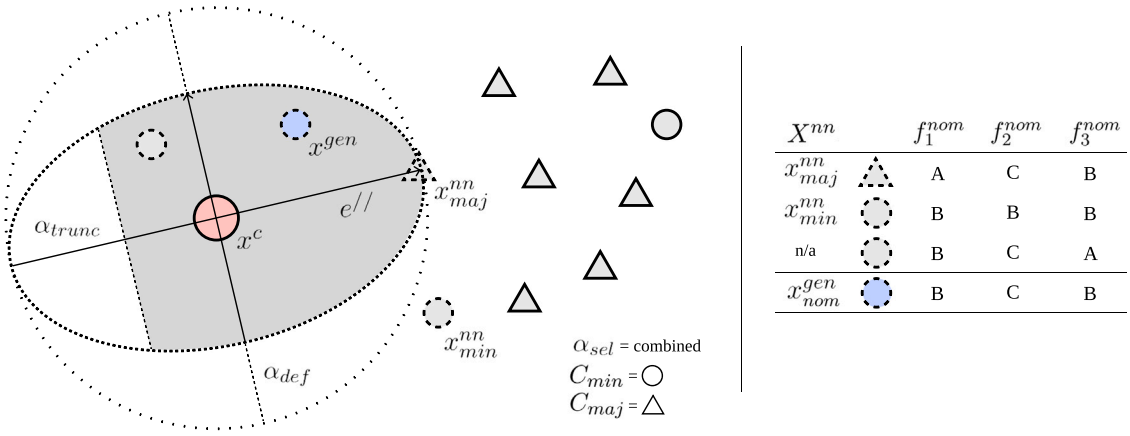
The data selection mechanism is preceded by the numerical encoding of the nominal features. It combines the selection mechanisms of SMOTENC and G-SMOTE, as shown in Algorithm 2. The selection mechanism inherits the minority, majority, and combined mechanisms proposed in G-SMOTE. The nominal features in the minority and majority class observations, C_{maj} and C_{min} are first encoded using a one-hot encoding approach and replacing the constant 1 with the median of the standard deviations of the continuous features in C_{min} divided by 2. The nearest-neighbors (X^{nn}) of x^c are determined based on α_{sel}, which are passed on to the generation mechanism to determine the nominal features' values of x^{gen} in the generation mechanism. Simultaneously, x^{nn} is randomly selected from X^{nn} and will be used to generate x^{gen}'s continuous features' values.

Algorithm 2: G-SMOTENC's selection mechanism.

Input: $C_{maj}, C_{min}, \alpha_{sel}$
Output: x^c, x^{nn}, X^{nn}
Function *CatEncoder*(C_{maj}, C_{min})**:**
  $S \leftarrow$ Standard deviations of the continuous features in C_{min}
  $\sigma_{med} \leftarrow median(S)$
  forall $i \in \{maj, min\}$ **do**
    forall $f \in C_i^T$ **do**
      if *f is nominal* **then**
        $f' \leftarrow OneHotEncode(f) \times \sigma_{med}/2$
        $C_i' \leftarrow (C_i^T \setminus f)^T$
        $C_i' \leftarrow (C_i'^T \cup f')^T$
  return C'_{maj}, C'_{min}
Function *Surface*(α_{sel}, x^c, C_{maj}, C_{min})**:**
  if $\alpha_{sel} = minority$ **then**
    $x^{nn} \in C_{min,k}$ `// One of the k-nearest neighbors of` x^c `from` C_{min}
    $X^{nn} \leftarrow C_{min,k}$
  if $\alpha_{sel} = majority$ **then**
    $x^{nn} \in C_{maj,1}$ `// Nearest neighbor of` x^c `from` C_{maj}
    $X^{nn} \leftarrow C_{maj,1}$
  if $\alpha_{sel} = combined$ **then**
    $x^{nn}_{min} \in C_{min,k}$
    $x^{nn}_{maj} \in C_{maj,1}$
    $x^{nn} \leftarrow argmin(||x^{nn}_{min} - x^c||, ||x^{nn}_{maj} - x^c||)$
    $X^{nn} \leftarrow C_{min,k} \cup C_{maj,1}$
  return x^{nn}, X^{nn} `//` X^{nn} `is the set of k-nearest neighbors`
begin
  $C'_{maj}, C'_{min} \leftarrow CatEncoder(C_{maj}, C_{min})$
  $x^c \in C'_{min}$ `// Randomly select` x^c `from` C'_{min}
  $x^{nn}, X^{nn} \leftarrow Surface(\alpha_{sel}, x^c, C'_{maj}, C'_{min})$
  Reverse encoding of nominal features in x^c, x^{nn} and X^{nn}

3.2. Generation mechanism

G-SMOTENC's generation mechanism is shown in Algorithm 3. It divides the generation of x^{gen} into two parts: (1) generation of continuous feature values and (2) generation of nominal feature values.



X^{nn}		f_1^{nom}	f_2^{nom}	f_3^{nom}
x^{nn}_{maj}	△	A	C	B
x^{nn}_{min}	○	B	B	B
n/a	○	B	C	A
x^{gen}_{nom}	●	B	C	B

Fig. 1. A visual depiction of G-SMOTENC. In this example, α_{trunc} is approximately 0.5 and α_{def} is approximately 0.4.

First, the nominal features from x^c and x^{nn} are discarded. Afterward, the continuous features are generated using G-SMOTE's generation mechanism; within a hyper-spheroid defined with α_{trunc} and α_{def}, which allows the non-linear generation of synthetic observations between x^c and x^{nn}. Finally, the nominal feature values are generated by the mode of each feature within the observations in X^{nn}.

Algorithm 3: G-SMOTENC's generation mechanism.

Input: $x^c, x^{nn}, X^{nn}, \alpha_{trunc}, \alpha_{def}$
Output: x^{gen}
Function *Hyperball()*:
  $v_i \sim \mathcal{N}(0,1)$
  $r \sim \mathcal{U}(0,1)$
  $x^{gen} \leftarrow r^{1/p} \frac{(v_1,\ldots,v_p)}{||(v_1,\ldots,v_p)||}$
  return x^{gen}
Function *Vectors*(x^c, x^{nn}, x^{gen}):
  $e^{//} \leftarrow \frac{x^{nn}-x^c}{||x^{nn}-x^c||}$
  $x^{//} \leftarrow (x^{gen} \cdot e^{//})e^{//}$
  $x^{\perp} \leftarrow x^{gen} - x^{//}$
  return $x^{//}, x^{\perp}$
Function *Truncate*($x^c, x^{nn}, x^{gen}, x^{//}, \alpha_{trunc}$):
  if $|\alpha_{trunc} - x^{//}| > 1$ **then**
    $x^{gen} \leftarrow x^{gen} - 2x^{//}$
  return x^{gen}
Function *Deform*($x^{gen}, x^{\perp}, \alpha_{def}$):
  return $x^{gen} - \alpha_{def} x^{\perp}$
Function *Translate*(x^c, x^{gen}, R):
  return $x^c + Rx^{gen}$
Function *GenNominal*(X^{nn}):
  $x^{gen}_{nom} = \emptyset$
  forall $f \in (X^{nn})^T$ **do**
    if *f is nominal* **then**
      $x^{gen}_{nom} \cup \{mode(f)\}$ `// Ties are decided with random selection`
  return x^{gen}_{nom}
begin
  Discard nominal features from x^c and x^{nn}
  $x^{gen} \leftarrow Hyperball()$
  $x^{//}, x^{\perp} \leftarrow Vectors(x^c, x^{nn}, x^{gen})$
  $x^{gen} \leftarrow Truncate(x^c, x^{nn}, x^{gen}, x^{//}, \alpha_{trunc})$
  $x^{gen} \leftarrow Deform(x^{gen}, x^{\perp}, \alpha_{def})$
  $x^{gen} \leftarrow Translate(x^c, x^{gen}, ||x^{nn}_{cont} - x^c||)$
  $x^{gen}_{nom} \leftarrow GenNominal(X^{nn})$
  $x^{gen} \leftarrow x^{gen} \cup x^{gen}_{nom}$

4. Methodology

This section describes how the evaluation of G-SMOTENC was performed. We describe the datasets used in the experiment, their source and preprocessing steps executed in Section 4.1. The resampling and classification methods used to analyze G-SMOTE's performance are listed in Section 4.2. The performance metrics used are defined in Section 4.3. Finally, the experimental procedure is described in Section 4.4.

4.1. Experimental data

The datasets used in this experiment were extracted from the UC Irvine Machine Learning Repository. All of the datasets are publicly available and cover a range of different domains. The criteria to select the datasets ensured that all datasets are imbalanced and contained non-metric features (*i.e.*, ordinal, nominal or binary). These datasets are used to show how the performance of different classifiers varies across over/undersamplers. The Abalone dataset (Clark, Schreter, & Adams, 1996) consists on determining the age of abalone based on physical measurements. The Adult dataset (Kohavi et al., 1996) was extracted from USA's 1994 Census database and its prediction task is to determine whether a person's income is over 50 K a year. The Annealling dataset (Quinlan, 1987a) contains 4 target classes regarding the modification of the physical and/or chemical properties of different materials. The Census-Income dataset was extracted from the 1994 and 1995 current population surveys conducted by the U.S. Census Bureau. The Contraceptive dataset (Lim, Loh, & Shih, 2000) is a subset of the 1987 National Indonesia Contraceptive Prevalence Survey, where the classification task is to predict the contraceptive method choice among married, not pregnant women. The Covertype dataset (Blackard & Dean, 1999) contains cartographic data on four wilderness areas in the Roosevelt National Forest of northern Colorado. The task is to predict the forest cover type. The Credit Approval dataset (Quinlan, 1987b) contains information on anonymized credit card applications. The classification task is to predict whether an applicant is approved to receive a credit card. The German Credit dataset contains customer information to determine their credit risk (either good or bad). The Heart Disease dataset (Detrano et al., 1989) is the merging of databases from four different healthcare institutions from Cleveland, Hungary, Switzerland and Long Beach. The classification task is to predict the presence of a heart disease in the patient.

All datasets were initially preprocessed manually with minimal manipulations. We removed features and/or observations with missing values and identified the non-metric features. The second stage of preprocessing was done systematically. It starts with the generation of artificially imbalanced datasets with different Imbalance Ratios ($IR =$

Table 1
Description of the datasets collected after data preprocessing. The sampling strategy is similar across datasets. Legend: (IR) Imbalance Ratio.

Dataset	Metric	Non-Metric	Obs.	Min. obs.	Maj. obs.	IR	Classes
Abalone	7	1	4139	15	689	45.93	18
Adult	6	8	5000	1268	3732	2.94	2
Adult (10)	6	8	5000	451	4549	10.09	2
Annealing	6	4	790	34	608	17.88	4
Census	7	24	5000	337	4663	13.84	2
Contraceptive	5	4	1473	333	629	1.89	3
Contraceptive (10)	5	4	1036	62	629	10.15	3
Contraceptive (20)	5	4	990	31	629	20.29	3
Contraceptive (31)	5	4	973	20	629	31.45	3
Contraceptive (41)	5	4	966	15	629	41.93	3
Covertype	10	2	5000	20	2449	122.45	7
Credit Approval	6	9	653	296	357	1.21	2
German Credit	7	13	1000	300	700	2.33	2
German Credit (10)	7	13	770	70	700	10.00	2
German Credit (20)	7	13	735	35	700	20.00	2
German Credit (30)	7	13	723	23	700	30.43	2
German Credit (41)	7	13	717	17	700	41.18	2
Heart Disease	5	5	740	22	357	16.23	5
Heart Disease (21)	5	5	735	17	357	21.00	5

$\frac{|C_{maj}|}{|C_{min}|}$). For each original dataset, we create its more imbalanced versions at intervals of 10, while ensuring that $|C_{min}| \geq 15$. The sampling strategy was determined for class $n \in \{1, \ldots, n, \ldots, m\}$ as a linear interpolation using $|C_{maj}|$ and $|C'_{min}| = \frac{|C_{maj}|}{IR_{new}}$, as shown in Eq. (1).

$$|C_i|^{imb} = \min(\frac{|C'_{min}| - |C_{maj}|}{n-1}.|C_i| + |C_{max}|, |C_i|) \quad (1)$$

The new, artificially imbalanced dataset, is formed by sampling observations without replacement from each C_i such that $C'_i \subseteq C_i, |C'_i| = |C_i|^{imb}$. The artificially imbalanced datasets are marked with its imbalance ratio as a suffix in Table 1.

The datasets (both original and artificially imbalanced versions) are then filtered to ensure all datasets have a minimum of 500 observations. The remaining datasets with a number of observations larger than 5000 are randomly sampled to match this number of observations. Afterward, we remove target classes with a frequency lower than 15 observations for each remaining dataset. Finally, the continuous and discrete features are scaled to the range $[0, 1]$ to ensure a common range between all features. The description of the resulting datasets is shown in Table 1.

4.2. Machine learning algorithms

The choice of classifiers used in the experimental procedure was based on their type (tree-based, nearest neighbors-based, linear model and ensemble-based), popularity and consistency in performance. We used Decision Tree (DT), a K-Nearest Neighbors (KNN) classifier, a Logistic Regression (LR) and a Random Forest (RF). Our choice of classifiers ensures an unbiased and accurate analysis of the results obtained in the experimental phase, based on the nature and popularity of these classifiers. However, it is important to note that the classifier has no influence on G-SMOTENC's resampling phase; the classifier is trained using the resampled (balanced) data, but does not affect the synthetic data generation process.

Given the lack of existing oversamplers that address imbalanced learning problems with mixed data types, the amount of benchmark methods used is also limited. We used three appropriate, well-known methods and one state-of-the-art oversampling method: SMOTENC, RUS, ROS and SMOTE-ENC. Table 2 shows the hyperparameters used for the parameter search described in Section 4.4.

4.3. Performance metrics

The choice of the performance metric plays a critical role in assessing the effect on classification tasks. The typical performance metrics, *e.g.*, Overall Accuracy (OA), are intuitive to interpret but are often inappropriate to measure a classifier's performance in an imbalanced learning context (Sun, Wong, & Kamel, 2009). For example, to estimate an event that occurs in 1% of the dataset, a constant classifier would obtain an OA of 0.99 and still be unusable. However, this metric is still reported in some of our results to maintain interpretability.

Table 2
Hyperparameter definition for the classifiers and resamplers used in the experiment.

Classifier	Hyperparameter	Values
DT	min. samples split	2
	criterion	gini
	max depth	3, 6
LR	maximum iterations	10000
	multi-class	One-vs-All
	solver	saga
	penalty	None, L1, L2
KNN	# neighbors	3, 5
	weights	uniform
	metric	euclidean
RF	min. samples split	2
	# estimators	50, 100
	Max depth	3, 6
	criterion	gini
Resampler		
SMOTENC	# neighbors	3, 5
SMOTE-ENC	# neighbors	3, 5
G-SMOTENC	# neighbors	3, 5
	deformation factor	0.0, 0.25, 0.5, 0.75, 1.0
	truncation factor	−1.0, −0.5, 0.0, 0.5, 1.0
	selection strategy	"combined", "minority", "majority"
RUS	replacement	False
ROS	(no applicable parameters)	

Recent surveys consider Geometric-mean (G-mean), F1-score (F-score), $Sensitivity = \frac{TP}{FN+TP}$ and $Specificity = \frac{TN}{TN+FP}$ appropriate and common performance metrics in imbalanced learning contexts (Japkowicz, 2013; Jeni, Cohn, & De La Torre, 2013; Rout, Mishra, & Mallick, 2018). G-mean and F-score are defined Eqs. (2) and (3), respectively.

$$G\text{-}mean = \sqrt{\overline{Sensitivity} \times \overline{Specificity}} \quad (2)$$

$$F\text{-}score = 2 \times \frac{\overline{Precision} \times \overline{Recall}}{\overline{Precision} + \overline{Recall}} \quad (3)$$

They are calculated as a function of the number of False/True Positives (FP and TP) and False/True Negatives (FN and TN), with $Precision = \frac{TP}{TP+FP}$ and $Recall = \frac{TP}{TP+FN}$. This led us to use, along with OA, both F-score and G-mean as the main performance metrics for this study.

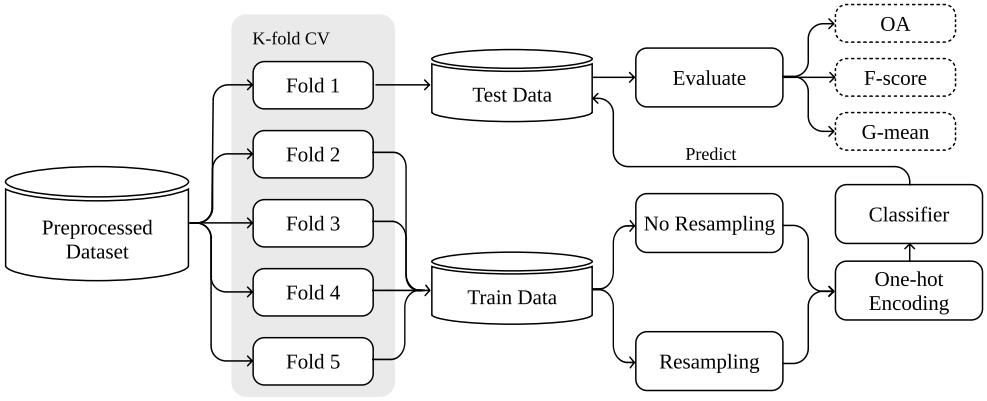


Fig. 2. Experimental procedure used in this study.

4.4. Experimental procedure

The experimental procedure was applied similarly to all combinations of resamplers, classifiers and hyperparameter combinations across all datasets. The evaluation of the models' performance was tested using a 5-fold Cross-Validation (CV) approach. The mean performance in the test set is calculated over the five folds and three different runs of the experimental procedure for each combination of resampling/classifier hyperparameters. For each dataset, we select the results of the hyperparameters that optimize the performance of a resampler/classifier. Fig. 2 shows a diagram of the experimental procedure described.

A CV run consists of a stratified partitioning (*i.e.*, each partition contains the same relative frequencies of target labels) of the dataset into five parts. A given resampler/classifier combination with a specific set of hyperparameters is fit and tested five times, using one of the partitions as a test set and the remaining ones as the training set. In the ML pipeline defined for each run, the nominal features are one-hot encoded after oversampling and before passing the data to the classifier. The estimated performance consists of the average classification performance across the five tests and three runs (*i.e.*, a total of 15 tests).

4.5. Software implementation

The algorithmic implementation of G-SMOTENC was written using the Python programming language and is available in the open-source package ML-Research (Fonseca, Douzas, & Bacao, 2021b), along with other utilities used to produce the experiment and outputs used in Section 5. In addition, the packages Scikit-Learn (Pedregosa et al., 2011), Imbalanced-Learn (Lemaître, Nogueira, & Aridas, 2017) and Research-Learn were also used in the experimental procedure to get the implementations of the classifiers, benchmark over/undersamplers and run the experimental procedure. The original SMOTE-ENC implementation was retrieved from the authors' GitHub repository. The Latex code, Python scripts (including data pulling and preprocessing, experiment setup and analysis of results), as well as the datasets used, are available in this GitHub repository.

5. Results and discussion

In this section, we present the experimental results. We focus on the comparison of classification performance using oversamplers whose generation mechanism is compatible with datasets containing both nominal and continuous features. The experimental results were analyzed in two stages: (1) in Section 5.1 we analyze mean rankings and absolute performances and in Section 5.2 we show the results of our statistical analysis. Section 5.3 discusses the main insights extracted by analyzing the experimental results.

5.1. Results

Table 3 presents the mean rankings of CV scores between the different combinations of oversamplers, metrics and classifiers. These results were calculated by assigning a ranking score for each oversampler from 1 (best) to 4 (worst) for each dataset, metric and classifier.

Table 4 presents the mean CV scores. Except for the OA metric, G-SMOTENC either outperformed or matched the remaining oversamplers.

Table 5 shows the mean and standard error of the percentage difference between G-SMOTENC and SMOTENC.

5.2. Statistical analysis

It is necessary to use methods that account for the multiple comparison problem to conduct an appropriate statistical analysis in an experiment with multiple datasets. Based on the recommendations found in Demšar (2006), we applied a Friedman test followed by a Holm–Bonferroni test for post-hoc analysis.

In Section 4.3 we explained that OA, although easily interpretable, is not an appropriate performance metric for imbalanced learning problems. Therefore, the statistical analysis was developed using the two imbalance-appropriate metrics used in the study: F-Score and G-Mean. Based on the Friedman test (Friedman, 1937), there is a statistically significant difference in performance across resampling methods. The results of this test are shown in Table 6. The null hypothesis is rejected in all cases.

We performed a Holm–Bonferroni test to understand whether the difference in the performance of G-SMOTENC is statistically significant to the remaining resampling methods. The results of this test are shown in Table 7. The null hypothesis is rejected in 33 out of 40 trials.

5.3. Discussion

The results reported in Section 5.1 show that G-SMOTENC consistently outperforms the remaining oversampling approaches. Based on the two metrics appropriate for imbalanced learning problems, G-Mean and F-Score, in the average rankings shown in Table 3 G-SMOTENC was only outperformed once by a small margin. Unlike the results reported in Mukherjee and Khushi (2021), SMOTE-ENC's performance was rarely superior to SMOTENC's.

The relative difference in the classifiers' performance is better visible in Table 4. Using an RF classifier, for example, the impact of using G-SMOTENC compared to no oversampling improves, on average, 13 percentual points on G-mean and nine percentual points using F-Score. The mean percentage difference shown in Table 5 shows that, on average, G-SMOTENC is always superior to SMOTENC.

The difference in performance between oversamplers was found to be statistically significant across classifiers and performance metrics in

Table 3
Mean rankings over the different datasets, folds and runs used in the experiment.

Classifier	Metric	G-SMOTENC	NONE	SMOTENC	ROS	RUS	SMOTE-ENC
DT	OA	1.66 ± 0.13	**1.61 ± 0.27**	3.58 ± 0.20	4.68 ± 0.15	5.42 ± 0.27	4.05 ± 0.23
DT	F-Score	**1.32 ± 0.11**	3.84 ± 0.40	3.13 ± 0.20	4.32 ± 0.19	5.47 ± 0.23	2.92 ± 0.34
DT	G-Mean	**1.68 ± 0.24**	5.84 ± 0.09	2.82 ± 0.21	2.95 ± 0.32	4.26 ± 0.32	3.45 ± 0.30
KNN	OA	2.50 ± 0.17	**1.37 ± 0.28**	4.21 ± 0.25	3.34 ± 0.35	5.68 ± 0.22	3.89 ± 0.15
KNN	F-Score	**1.37 ± 0.16**	3.95 ± 0.35	3.11 ± 0.29	3.47 ± 0.36	5.53 ± 0.23	3.58 ± 0.23
KNN	G-Mean	**1.74 ± 0.17**	5.84 ± 0.12	2.89 ± 0.23	3.76 ± 0.33	3.00 ± 0.45	3.76 ± 0.23
LR	OA	2.74 ± 0.19	**1.37 ± 0.28**	3.08 ± 0.21	4.34 ± 0.30	5.74 ± 0.17	3.74 ± 0.28
LR	F-Score	**2.11 ± 0.24**	4.53 ± 0.35	2.37 ± 0.28	3.47 ± 0.32	5.21 ± 0.27	3.32 ± 0.38
LR	G-Mean	2.13 ± 0.26	6.00 ± 0.00	3.61 ± 0.21	**2.11 ± 0.23**	3.32 ± 0.40	3.84 ± 0.28
RF	OA	1.82 ± 0.11	**1.24 ± 0.09**	3.97 ± 0.16	4.32 ± 0.21	5.92 ± 0.06	3.74 ± 0.22
RF	F-Score	**1.32 ± 0.13**	5.05 ± 0.31	3.16 ± 0.22	3.05 ± 0.31	5.37 ± 0.14	3.05 ± 0.27
RF	G-Mean	**1.68 ± 0.22**	5.79 ± 0.21	3.26 ± 0.28	2.47 ± 0.30	3.89 ± 0.35	3.89 ± 0.19

Table 4
Mean scores over the different datasets, folds and runs used in the experiment.

Classifier	Metric	G-SMOTENC	NONE	SMOTENC	ROS	RUS	SMOTE-ENC
DT	OA	0.74 ± 0.05	**0.75 ± 0.04**	0.68 ± 0.04	0.66 ± 0.04	0.58 ± 0.04	0.65 ± 0.04
DT	F-Score	**0.56 ± 0.04**	0.52 ± 0.04	0.54 ± 0.04	0.52 ± 0.04	0.48 ± 0.04	0.51 ± 0.04
DT	G-Mean	**0.69 ± 0.03**	0.60 ± 0.02	0.68 ± 0.03	0.67 ± 0.03	0.65 ± 0.03	0.66 ± 0.03
KNN	OA	0.69 ± 0.04	**0.73 ± 0.05**	0.67 ± 0.04	0.69 ± 0.05	0.57 ± 0.04	0.68 ± 0.05
KNN	F-Score	**0.53 ± 0.04**	0.50 ± 0.04	0.52 ± 0.04	0.52 ± 0.04	0.46 ± 0.04	0.51 ± 0.04
KNN	G-Mean	**0.66 ± 0.03**	0.58 ± 0.03	0.64 ± 0.03	0.62 ± 0.03	0.65 ± 0.03	0.63 ± 0.03
LR	OA	0.68 ± 0.05	**0.75 ± 0.04**	0.68 ± 0.05	0.66 ± 0.05	0.58 ± 0.04	0.67 ± 0.04
LR	F-Score	**0.54 ± 0.04**	0.52 ± 0.04	**0.54 ± 0.04**	0.53 ± 0.04	0.48 ± 0.04	0.52 ± 0.04
LR	G-Mean	**0.69 ± 0.02**	0.60 ± 0.03	0.68 ± 0.02	**0.69 ± 0.03**	0.67 ± 0.03	0.67 ± 0.03
RF	OA	0.74 ± 0.04	**0.76 ± 0.04**	0.69 ± 0.04	0.69 ± 0.04	0.59 ± 0.04	0.68 ± 0.05
RF	F-Score	**0.57 ± 0.04**	0.48 ± 0.04	0.55 ± 0.04	0.55 ± 0.04	0.49 ± 0.04	0.53 ± 0.04
RF	G-Mean	**0.70 ± 0.02**	0.57 ± 0.02	0.68 ± 0.03	0.69 ± 0.03	0.68 ± 0.03	0.68 ± 0.02

Table 5
Percentage difference between G-SMOTENC and SMOTE across all datasets.

Classifier	Metric	Difference
DT	OA	7.68 ± 1.12
DT	F-Score	4.27 ± 0.85
DT	G-Mean	2.62 ± 0.68
KNN	OA	3.29 ± 0.82
KNN	F-Score	3.10 ± 0.66
KNN	G-Mean	3.09 ± 0.95
LR	OA	0.32 ± 0.23
LR	F-Score	0.17 ± 0.21
LR	G-Mean	1.49 ± 0.31
RF	OA	7.03 ± 1.30
RF	F-Score	5.20 ± 0.83
RF	G-Mean	2.77 ± 0.84

Table 6
Results for the Friedman test. Statistical significance is tested at a level of $\alpha = 0.05$. The null hypothesis is that there is no difference in the classification outcome across resamplers.

Classifier	Metric	*p*-value	Significance
DT	F-Score	2.2e−10	True
DT	G-Mean	1.2e−10	True
KNN	F-Score	2.3e−09	True
KNN	G-Mean	9.4e−10	True
LR	F-Score	2.1e−07	True
LR	G-Mean	9.7e−11	True
RF	F-Score	8.5e−12	True
RF	G-Mean	2.0e−10	True

Table 7
Adjusted p-values using the Holm–Bonferroni test. Statistical significance is tested at a level of $\alpha = 0.05$. The null hypothesis is that the benchmark methods perform similarly to the control method (G-SMOTENC).

Classifier	Metric	NONE	SMOTENC	ROS	RUS	SMOTE-ENC
DT	F-Score	**1.5e−04**	**1.5e−04**	**7.3e−06**	**1.2e−06**	1.0e−01
DT	G-Mean	**5.6e−07**	**2.7e−03**	**2.8e−02**	**3.9e−04**	**2.3e−02**
KNN	F-Score	**6.4e−04**	**2.2e−04**	**7.2e−04**	**6.4e−04**	**5.9e−06**
KNN	G-Mean	**1.6e−05**	**9.6e−03**	**6.5e−03**	2.0e−01	**3.5e−03**
LR	F-Score	**4.0e−03**	6.1e−01	**9.2e−03**	**3.6e−04**	5.6e−02
LR	G-Mean	**1.6e−07**	**4.0e−04**	8.6e−01	2.4e−01	**4.7e−03**
RF	F-Score	**1.7e−06**	**2.4e−04**	**8.0e−03**	**1.7e−06**	**8.0e−03**
RF	G-Mean	**3.8e−06**	**8.8e−03**	2.5e−01	**2.3e−02**	**1.7e−03**

the Friedman test. The p-values of this test are reported in Table 6. The superiority of G-SMOTENC was confirmed with the results from the Holm–Bonferroni test shown in Table 7. This test showed that G-SMOTENC outperformed with statistical significance the remaining resamplers in 82.5% of the comparisons done.

The results from this experiment expose some well-known limitations of SMOTE, which become particularly evident with SMOTENC. Specifically, the lack of diversity in the generated data and, on some occasions, the near-duplication of observations discussed in Douzas and Bacao (2019) may be a possible explanation for the performance of SMOTENC being comparable to ROS' performance, visible in Fig. 3. In this figure, three groups of resampling methods with comparable performance are visible: (1) G-SMOTENC, the top-performing method, (2) SMOTENC, ROS and SMOTE-ENC, where SMOTE-ENC has the most inconsistent behavior and (3) RUS and no oversampling, the worst-performing approaches. In addition, G-SMOTENC's superiority seems invariable to the dataset's characteristics, with little overlap with the remaining benchmark methods.

6. Conclusion

This paper presented G-SMOTENC, a new oversampling algorithm that combines G-SMOTE and SMOTENC. This oversampling algorithm leverages G-SMOTE's data selection and generation mechanisms into datasets with mixed data types. This was achieved by encoding and generating nominal feature values using SMOTENC's approach. The quality of the data generated with G-SMOTENC was tested over 20 datasets with different imbalance ratios, metric/non-metric feature ratios and number of classes. These results were compared to no oversampling, SMOTENC, Random Oversampling, Random Undersampling and SMOTE-ENC using a Decision Tree, K-Nearest Neighbors, Logistic Regression and Random Forest as classifiers.

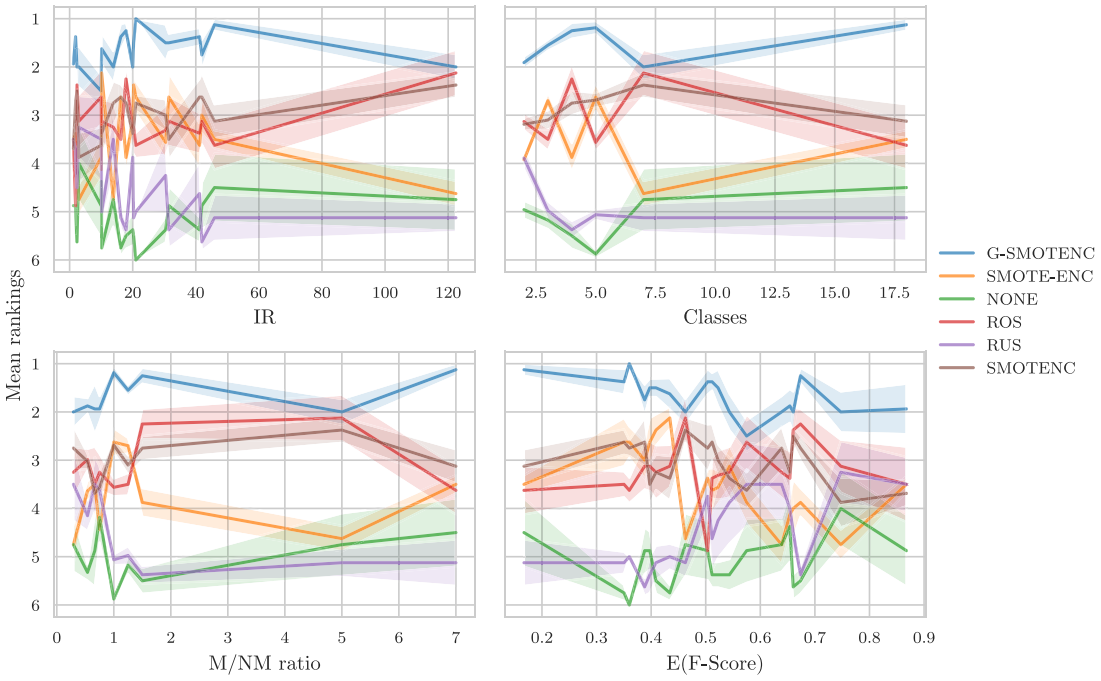


Fig. 3. Average ranking of oversamplers over different characteristics of the datasets used in the experiment.
Legend: IR — Imbalance Ratio, Classes — Number of classes in the dataset, M/NM ratio — ratio between the number of metric and non-metric features, E(F-Score) — Mean F-Score of dataset across all combinations of classifiers and oversamplers.

G-SMOTENC can be seen as a drop-in replacement of SMOTENC, since when $\alpha_{trunc} = 1$, $\alpha_{def} = 1$ and $\alpha_{sel} = minority$, SMOTENC is reproduced. G-SMOTENC has three additional hyperparameters that allow for greater customization of the selection and generation mechanisms. However, determining the optimal parameters a priori (*i.e.*, with reduced parameter tuning) is a topic for future work.

The results show that G-SMOTENC performs significantly better when compared to its more popular counterparts (SMOTENC, Random Oversampling and Random Undersampling), as well as a recently proposed oversampling algorithm for mixed data types (SMOTE-ENC). This performance improvement is related to G-SMOTENC's selection mechanism, which finds a safer region for data generation, along with its generation mechanism which increases the diversity of the generated observations compared to SMOTENC. The G-SMOTENC implementation used in this study is available in the open-source Python library "ML-Research" and is fully compatible with the Scikit-Learn ecosystem.

Funding

This research was supported by research grants of the Portuguese Foundation for Science and Technology ("Fundação para a Ciência e a Tecnologia"), references SFRH/BD/151473/2021, DSAIPA/DS/0116/2019, and by project UIDB/04152/2020 — Centro de Investigação em Gestão de Informação (MagIC).

CRediT authorship contribution statement

Joao Fonseca: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing, Formal analysis, Data curation. **Fernando Bacao:** Conceptualization, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The Latex code, Python scripts (including data pulling and preprocessing, experiment setup and analysis of results), as well as the datasets are open sourced (link provided in Section 4.5)

References

Ambai, K., & Fujita, H. (2018). MNDO: Multivariate normal distribution based oversampling for binary classification. In *Advancing technology industrialization through intelligent software methodologies, tools and techniques: Proceedings of the 17th international conference on new trends in intelligent software methodologies, tools and techniques* (pp. 425–438).

Ambai, K., & Fujita, H. (2019). Multivariate normal distribution based over-sampling for numerical and categorical features. In *Advancing technology industrialization through intelligent software methodologies, tools and techniques: Proceedings of the 18th international conference on new trends in intelligent software methodologies, tools and techniques (SoMeT), Vol. 318* (p. 107).

Bansal, A., & Jain, A. (2021). Analysis of focussed under-sampling techniques with machine learning classifiers. In *2021 IEEE/ACIS 19th international conference on software engineering research, management and applications* (pp. 91–96). IEEE.

Batista, G. E., Prati, R. C., & Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter*, *6*(1), 20–29.

Blackard, J. A., & Dean, D. J. (1999). Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*, *24*(3), 131–151.

Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2009). Safe-level-smote: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Pacific-Asia conference on knowledge discovery and data mining* (pp. 475–482). Springer.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 321–357.

Clark, D., Schreter, Z., & Adams, A. (1996). A quantitative comparison of dystal and backpropagation. In *Australian conference on neural networks* (pp. 132–137).

Das, S., Datta, S., & Chaudhuri, B. B. (2018). Handling data irregularities in classification: Foundations, trends, and future challenges. *Pattern Recognition*, *81*, 674–693.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*, 1–30.

Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J. J., Sandhu, S., et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, *64*(5), 304–310.

Douzas, G., & Bacao, F. (2019). Geometric SMOTE a geometrically enhanced drop-in replacement for SMOTE. *Information Sciences*, *501*, 118–135.

Douzas, G., Bacao, F., Fonseca, J., & Khudinyan, M. (2019). Imbalanced learning in land cover classification: Improving minority classes' prediction accuracy using the geometric SMOTE algorithm. *Remote Sensing*, *11*(24), 3040.

Douzas, G., Bacao, F., & Last, F. (2018). Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE. *Information Sciences*, *465*, 1–20.

Fernández, A., Garcia, S., Herrera, F., & Chawla, N. V. (2018). SMOTE for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. *Journal of Artificial Intelligence Research*, *61*, 863–905.

Fernández, A., LóPez, V., Galar, M., Del Jesus, M. J., & Herrera, F. (2013). Analysing the classification of imbalanced data-sets with multiple classes: Binarization techniques and ad-hoc approaches. *Knowledge-Based Systems*, *42*, 97–110.

Fonseca, J., Douzas, G., & Bacao, F. (2021a). Improving imbalanced land cover classification with K-means SMOTE: Detecting and oversampling distinctive minority spectral signatures. *Information*, *12*(7), 266.

Fonseca, J., Douzas, G., & Bacao, F. (2021b). Increasing the effectiveness of active learning: Introducing artificial data generation in active learning for land use/land cover classification. *Remote Sensing*, *13*(13), 2619.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, *32*(200), 675–701.

Gonog, L., & Zhou, Y. (2019). A review: generative adversarial networks. In *2019 14th IEEE conference on industrial electronics and applications* (pp. 505–510). IEEE.

Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing* (pp. 878–887). Springer.

He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)* (pp. 1322–1328). IEEE.

Japkowicz, N. (2013). Assessment metrics for imbalanced learning. *Imbalanced Learning: Foundations, Algorithms, and Applications*, 187–206.

Jeni, L. A., Cohn, J. F., & De La Torre, F. (2013). Facing imbalanced data–recommendations for the use of performance metrics. In *2013 Humaine association conference on affective computing and intelligent interaction* (pp. 245–251). IEEE.

Jo, W., & Kim, D. (2022). OBGAN: Minority oversampling near borderline with generative adversarial networks. *Expert Systems with Applications*, *197*, Article 116694.

Kaur, H., Pannu, H. S., & Malhi, A. K. (2019). A systematic review on imbalanced data challenges in machine learning: Applications and solutions. *ACM Computing Surveys*, *52*(4), 1–36.

Kohavi, R., et al. (1996). Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid.. In *Kdd, Vol. 96* (pp. 202–207).

Koivu, A., Sairanen, M., Airola, A., & Pahikkala, T. (2020). Synthetic minority oversampling of vital statistics data with generative adversarial networks. *Journal of the American Medical Informatics Association*, *27*(11), 1667–1674.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, *18*(17), 1–5.

Liang, X., Jiang, A., Li, T., Xue, Y., & Wang, G. (2020). LR-SMOTE—An improved unbalanced data set oversampling based on K-means and SVM. *Knowledge-Based Systems*, *196*, Article 105845.

Lim, T.-S., Loh, W.-Y., & Shih, Y.-S. (2000). A comparison of prediction accuracy, complexity, and training time of thirty-three old and new classification algorithms. *Machine Learning*, *40*, 203–228.

López, V., Fernández, A., García, S., Palade, V., & Herrera, F. (2013). An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics. *Information Sciences*, *250*, 113–141.

Lumijärvi, J., Laurikkala, J., & Juhola, M. (2004). A comparison of different heterogeneous proximity functions and euclidean distance. In *MEDINFO 2004* (pp. 1362–1366). IOS Press.

Mukherjee, M., & Khushi, M. (2021). SMOTE-ENC: A novel SMOTE-based method to generate synthetic data for nominal and continuous features. *Applied System Innovation*, *4*(1), 18.

Park, S., & Park, H. (2021). Combined oversampling and undersampling method based on slow-start algorithm for imbalanced network traffic. *Computing*, *103*(3), 401–424.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Quinlan, J. R. (1987a). Decision trees as probabilistic classifiers. In *Proceedings of the fourth international workshop on machine learning* (pp. 31–37). Elsevier.

Quinlan, J. R. (1987b). Simplifying decision trees. *International Journal of Man-Machine Studies*, *27*(3), 221–234.

Rout, N., Mishra, D., & Mallick, M. K. (2018). Handling imbalanced data: A survey. In *International proceedings on advances in soft computing, intelligent systems and applications* (pp. 431–443). Springer.

Salazar, A., Vergara, L., & Safont, G. (2021). Generative adversarial networks and Markov random fields for oversampling very small training sets. *Expert Systems with Applications*, *163*, Article 113819.

Sun, Y., Wong, A. K., & Kamel, M. S. (2009). Classification of imbalanced data: A review. *International Journal of Pattern Recognition and Artificial Intelligence*, *23*(04), 687–719.

Tarekegn, A. N., Giacobini, M., & Michalak, K. (2021). A review of methods for imbalanced multi-label classification. *Pattern Recognition*, *118*, Article 107965.

Tyagi, S., & Mittal, S. (2020). Sampling approaches for imbalanced data classification problem in machine learning. In *Proceedings of ICRIC 2019* (pp. 209–221). Springer.

Vuttipittayamongkol, P., Elyan, E., & Petrovski, A. (2021). On the class overlap problem in imbalanced data classification. *Knowledge-Based Systems*, *212*, Article 106631.